

EFConventionBuilder

Developer & AI Context Reference — v2.3

Convention-over-configuration for EF Core. For use as context in new development sessions and AI chat.

Library Overview

EFConventionBuilder is a NuGet package (**EFConventionBuilder**) that scans a domain assembly at startup and configures EF Core automatically — tables, columns, FK relationships, soft delete query filters, and audit field mapping — without any explicit OnModelCreating boilerplate. Published on nuget.org. Source on GitHub at github.com/mtnman1010/EFConvention.

File	Purpose
IEntity.cs	IEntityBase and IEntity — discovery marker interfaces
IBehaviors.cs	IAuditable and ISoftDelete — behavioural contracts
Behaviors.cs	AuditColumnNames and SoftDeleteColumnNames — column name overrides
ICurrentUserService.cs	Identity abstraction — implemented once in the application
IEntityNamingConvention.cs	Naming convention interface — three built-in strategies
EntityConventionBuilder.cs	Core facade — fluent configuration and model building
IUnitOfWork.cs	Testable data access interface
UnitOfWork.cs	Abstract EF Core DbContext implementing IUnitOfWork
AuditInterceptor.cs	SaveChangesInterceptor that stamps IAuditable fields
ServiceBase.cs	Abstract service base — delete, restore, and purge helpers

Entity Discovery

All concrete classes implementing **IEntityBase** or **IEntity** in the target assembly are discovered automatically. Abstract classes and interfaces are skipped. No DbSet properties, [Table] attributes, or explicit registration required.

Interface	Purpose
IEntityBase	Root discovery marker. Any implementing class is registered as an EF entity.
IEntity : IEntityBase	Typed integer primary key. Most domain entities implement this directly.
IAuditable	Marks an entity for audit field stamping via AuditInterceptor.
ISoftDelete	Marks an entity for soft delete with a global query filter.
ICurrentUserService	Identity abstraction. Defined in library, implemented once per application.

Factory methods: `ForAssemblyOf<T>()` scans the assembly containing T. `ForAssembly(Assembly)` scans a specific assembly. `ForTypes(params Type[])` registers only the specified types — useful in tests and bounded contexts to avoid picking up unrelated types from the same assembly.

Naming Conventions

Method	Table	Column	FK column
default — PascalCase	Customer	OrderDate	Customer
<code>.UseSnakeCase()</code>	customer	order_date	customer
<code>.UsePluralizedTables()</code>	Customers	OrderDate	Customer
<code>.UseNamingConvention(custom)</code>	your choice	your choice	your choice

FK columns are named after the navigation property name — not the type with an Id suffix. `Order.Customer` → column named Customer (not CustomerId). This handles multiple navigations to the same type unambiguously: `Withholding.Person` → Person, `Withholding.Payee` → Payee, `Withholding.OnBehalfOf` → OnBehalfOf. Under snake_case: person, payee, on_behalf_of.

Multiple collections disambiguation (StartsWith convention): When a principal has multiple collections of the same dependent type, the builder matches the collection name to the navigation name using `StartsWith`. `PersonWithholdings` maps to the Person navigation, `PayeeWithholdings` maps to the Payee navigation, `OnBehalfOfWithholdings` maps to the OnBehalfOf navigation. Name collections starting with the corresponding navigation property name on the dependent.

Relationship Detection — Required vs Optional

Three-step priority chain — first matching rule wins:

Priority	Condition	Result
1	[Required] attribute on navigation property	IsRequired(true)
2	Scalar FK property int (non-nullable)	IsRequired(true)
2	Scalar FK property int? (nullable)	IsRequired(false)
3	Non-nullable navigation Address Address = null!	IsRequired(true)
3	Nullable navigation Address? Address	IsRequired(false)

Priority 3 — nullable reference type convention: Scalar FK properties (`CategoryId`, `CustomerId` etc.) are not required. The nullability of the navigation property itself determines required vs optional. Requires `<Nullable>enable</Nullable>` in the project file — without it all navigations are treated as optional. Scalar FK properties are still supported for backwards compatibility and for bulk operations where setting the FK without loading the navigation is needed.

EF Core shadow properties: When no scalar FK property exists, EF Core creates a shadow property named `{NavName}Id` internally. The library renames only the database column to the navigation property name. The shadow property name is never visible in domain code.

Querying optional navigations in LINQ: Guard before accessing properties — `.Where(r => r.Customer != null && r.Customer.Id == id)`. EF Core translates this to efficient SQL. Do not use delegate-based helpers (`NotNull` etc.) inside EF Core LINQ expressions — they cannot be translated to SQL. Delegate helpers are safe for in-memory evaluation on already-loaded entities (ViewModels, service logic).

IAuditable — Audit Stamping

`AuditInterceptor` (`SaveChangesInterceptor`) stamps `IAuditable` fields on every save. Registered via `AddInterceptors()` in the concrete `UnitOfWork` subclass. The `UnitOfWork` base class has no dependency on `ICurrentUserService` — that connection is made entirely in application code. Falls back to system when `ICurrentUserService.UserName` is null.

EntityState	Fields stamped
Added	CreatedDate =.UtcNow, CreatedBy = UserName
Modified	ModifiedDate =.UtcNow, ModifiedBy = UserName

C# property	SQL type	Nullable	Default column name
CreatedDate	DATETIME2	No	CreatedDate
CreatedBy	NVARCHAR(255)	No	CreatedBy
ModifiedDate	DATETIME2	Yes	ModifiedDate
ModifiedBy	NVARCHAR(255)	Yes	ModifiedBy

Column names can be overridden via `WithAuditFields(cols => ...)`. The active naming convention is applied on top of any override. Never set audit properties manually — `AuditInterceptor` handles them. Enabled by calling `.WithAuditFields()` or `.WithFullAudit()` on the builder.

ISoftDelete — Soft Delete

A global EF Core query filter (`WHERE IsDeleted = 0`) is registered automatically for every entity implementing `ISoftDelete` when `.WithSoftDelete()` or `.WithFullAudit()` is configured. Deleted rows are never physically removed — they are hidden from all normal queries. Use `.IgnoreQueryFilters()` to reach them.

C# property	SQL type	Nullable	Default column name
IsDeleted	BIT	No	IsDeleted — DEFAULT 0
DeletedDate	DATETIME2	Yes	DeletedDate
DeletedBy	NVARCHAR(255)	Yes	DeletedBy

Column names can be overridden via `WithSoftDelete(cols => ...)`. Never set `IsDeleted`, `DeletedDate`, or `DeletedBy` manually — use `ServiceBase.DeleteAsync`, `RestoreAsync`, and `PurgeAsync`.

Operation	IsDeleted	Row visible to normal query
AddAsync	false	Yes

DeleteAsync	true	No — hidden by global filter
RestoreAsync	false	Yes — restored
PurgeAsync	—	No — permanently deleted (soft delete required first)

IUnitOfWork and UnitOfWork

All application services depend on IUnitOfWork — not the concrete DbContext. This enables unit testing with Moq and loose coupling between service and persistence layers. UnitOfWork is an abstract EF Core DbContext that implements IUnitOfWork. Create one concrete subclass per database.

Method	Description
CompleteAsync(ct)	Flushes all change tracker entries to the database atomically.
Query<T>()	Returns IQueryable<T> for LINQ composition. Supports Include, IgnoreQueryFilters etc.
Add<T>(entity)	Stages an INSERT in the change tracker. No I/O until CompleteAsync.
Update<T>(entity)	Stages an UPDATE in the change tracker. No I/O until CompleteAsync.
Remove<T>(entity)	Stages a DELETE in the change tracker. No I/O until CompleteAsync.
FindAsync<T>(id, ct)	Looks up by PK. Returns cached instance if already tracked (ValueTask).
RefreshAsync<T>(…)	Reloads reference navigation properties from the database after save.
RefreshCollectionAsync(…)	Reloads a collection navigation property from the database after save.

Add, Update, and Remove are synchronous — they touch only EF Core's in-memory change tracker. All database I/O is deferred until CompleteAsync is called, committing all pending changes in a single transaction. FindAsync returns ValueTask because it is frequently synchronous when the entity is already tracked.

Assembly anchor: The domainAssembly parameter in the UnitOfWork constructor accepts any type from the domain assembly — it is just a scan anchor. It has no relationship to ICurrentUserService or audit stamping. Any stable domain type works.

ServiceBase

Abstract base for application services. TEntity is the primary entity type — it determines what DeleteAsync, RestoreAsync, and PurgeAsync operate on. It does not restrict which entity types the service can query or modify through UnitOfWork. Services should be organised around business capabilities, not entity types.

Method	Behaviour
DeleteAsync	Soft delete if TEntity implements ISoftDelete. Hard delete (Remove) if not. Detected at runtime — removing ISoftDelete
RestoreAsync	Clears IsDeleted, DeletedDate, DeletedBy. Row becomes visible again. Throws InvalidOperationException if TEntity
PurgeAsync	Permanently removes a soft-deleted row. Requires IsDeleted = true first (two-step pattern). Throws if entity is still acti

ICurrentUserService — Identity Abstraction

Required only when audit stamping is enabled via WithAuditFields() or WithFullAudit(). The interface is defined in the library. The implementation is provided once per application and varies by hosting environment. Falls back to system when Username returns null.

Hosting environment	Implementation approach
---------------------	-------------------------

ASP.NET Core MVC / API	IHttpContextAccessor → User.Identity.Name
Web API with JWT	Claims principal → NameIdentifier or sub claim
Blazor Server	AuthenticationStateProvider
WPF / WinForms	WindowsIdentity.GetCurrent().Name or custom session service
Background jobs	SystemUserService — always returns system
Console / CLI	Environment.UserName or prompt at startup
Unit tests	FixedUserService with a known username
No audit stamping needed	Do not implement — skip WithAuditFields() entirely

Per-Entity Opt-In Matrix

Entity implements	Builder configured with	Audit columns	Soft-delete column
neither	anything	No	No
IAuditable only	.WithAuditFields() / .WithFullAudit()	Yes	No
ISoftDelete only	.WithSoftDelete() / .WithFullAudit()	No	Yes
both	.WithFullAudit()	Yes	Yes

Complete Column Reference — PascalCase + WithFullAudit()

Store domain sample — seven entities covering every convention feature. FK columns use navigation property names. snake_case equivalents: apply underscore before uppercase following lowercase, then lowercase all.

Address — plain entity

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
Street	NVARCHAR(255)	No	
City	NVARCHAR(100)	No	
State	NVARCHAR(100)	No	
PostalCode	NVARCHAR(20)	No	

Category — plain entity

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
Name	NVARCHAR(100)	No	

Description	NVARCHAR(500)	No	
-------------	---------------	----	--

Customer — IAuditable, hard delete

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
Name	NVARCHAR(255)	No	
Email	NVARCHAR(255)	No	
Phone	NVARCHAR(50)	No	
Address	INT	No	FK → Address.Id — required (non-nullable nav)
CreatedDate	DATETIME2	No	Set on INSERT (UTC)
CreatedBy	NVARCHAR(255)	No	Set on INSERT
ModifiedDate	DATETIME2	Yes	Set on UPDATE (UTC), NULL until first update
ModifiedBy	NVARCHAR(255)	Yes	Set on UPDATE, NULL until first update

Product — IAuditable + ISoftDelete + DECIMAL(18,2)

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
Name	NVARCHAR(255)	No	
Description	NVARCHAR(1000)	No	
Sku	NVARCHAR(100)	No	
Price	DECIMAL(18,2)	No	[Precision(18,2)]
CostPrice	DECIMAL(18,2)	No	[Precision(18,2)]
Category	INT	No	FK → Category.Id — required (non-nullable nav)
CreatedDate	DATETIME2	No	Set on INSERT (UTC)
CreatedBy	NVARCHAR(255)	No	Set on INSERT
ModifiedDate	DATETIME2	Yes	Set on UPDATE (UTC)
ModifiedBy	NVARCHAR(255)	Yes	Set on UPDATE
IsDeleted	BIT	No	Default 0
DeletedDate	DATETIME2	Yes	Set on soft delete (UTC)
DeletedBy	NVARCHAR(255)	Yes	Set on soft delete

Order — IAuditable + ISoftDelete

Order is a reserved SQL Server keyword — use `dbo.[Order]` in raw SQL.

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
OrderDate	DATETIME2	No	
Status	NVARCHAR(50)	No	Default 'Pending'
TotalAmount	DECIMAL(18,2)	No	[Precision(18,2)]
Customer	INT	No	FK → Customer.Id — required (non-nullable nav)
CreatedDate	DATETIME2	No	Set on INSERT (UTC)
CreatedBy	NVARCHAR(255)	No	Set on INSERT
ModifiedDate	DATETIME2	Yes	Set on UPDATE (UTC)
ModifiedBy	NVARCHAR(255)	Yes	Set on UPDATE
IsDeleted	BIT	No	Default 0
DeletedDate	DATETIME2	Yes	Set on soft delete (UTC)
DeletedBy	NVARCHAR(255)	Yes	Set on soft delete

OrderItem — plain entity, required FKs

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
Quantity	INT	No	
UnitPrice	DECIMAL(18,2)	No	[Precision(18,2)]
Order	INT	No	FK → Order.Id — required (non-nullable nav)
Product	INT	No	FK → Product.Id — required (non-nullable nav)

ProductReview — IAuditable, optional Customer FK

Column	SQL type	Nullable	Notes
Id	INT IDENTITY	No	Primary key
Rating	INT	No	1–5
Comment	NVARCHAR(MAX)	No	
Product	INT	No	FK → Product.Id — required (non-nullable nav)
Customer	INT	Yes	FK → Customer.Id — optional (nullable nav Customer?)
CreatedDate	DATETIME2	No	Set on INSERT (UTC)

CreatedBy	NVARCHAR(255)	No	Set on INSERT
ModifiedDate	DATETIME2	Yes	Set on UPDATE (UTC)
ModifiedBy	NVARCHAR(255)	Yes	Set on UPDATE

Developer Checklist

- Nullable enable in project file — required for nullable reference type FK detection
- Collection navigation properties use private set; — public setter causes startup error
- Non-nullable navigation properties initialised with = null! to suppress compiler warning
- FK columns named after navigation property (Customer not CustomerId)
- Multiple collections of same type: collection name starts with navigation name (PersonWithHoldings)
- IAuditable: CreatedDate/CreatedBy NOT NULL, ModifiedDate/ModifiedBy NULL
- ISoftDelete: IsDeleted NOT NULL DEFAULT 0, DeletedDate/DeletedBy NULL
- Never set audit or soft delete fields manually — use AuditInterceptor and ServiceBase
- base.OnModelCreating(modelBuilder) called BEFORE _conventions.Apply(modelBuilder)
- Decimal precision properties carry [Precision(18,2)] attribute
- Partial index on each ISoftDelete table: WHERE IsDeleted = 0
- Order table bracket-quoted in raw SQL: dbo.[Order]
- AuditInterceptor registered via AddInterceptors() in UnitOfWork.OnConfiguring
- ICurrentUserService only required when WithAuditFields() or WithFullAudit() is used
- Assembly anchor (typeof(Customer).Assembly) is a scan anchor — unrelated to ICurrentUserService
- EnableServiceProviderCaching(false) on in-memory DbContext in tests
- ForTypes() used in tests to avoid picking up unrelated test fixture types

Potential Roadmap

Area	Description
Composite keys	Support for multi-column primary keys. Most impactful for junction/join tables (StudentCourse with StudentId and CourseId)
RegisterPreSaveTask	Queue actions that run atomically before or after SaveChanges — domain events, cache invalidation, bulk operations
Completing event	Fires before save with a dictionary of all changed entities and states. Flexible hook for domain event dispatch
Diagnostic output	.Describe() or .Validate() method returning a human-readable summary of all discovered entities, tables, columns
ForTypes exclusions	ForAssemblyOf<T>().Exclude<BadEntity>() — scan an assembly but skip specific types without switching to reflection
EFConvention.Domain	Optional companion NuGet package — pre-built domain building blocks (IHasNotes, IHasTags, IHasAttachments)
CHANGELOG.md	Formal changelog in the repo root. Low effort, high value for community.
CI workflow	GitHub Actions workflow triggering on PR and branch push — build and test without publish. Add as required

Configuration Patterns — Quick Reference

Pattern	Builder call
---------	--------------

PascalCase, no audit	EntityConventionBuilder.ForAssemblyOf<T>()
snake_case, no audit	.ForAssemblyOf<T>().UseSnakeCase()
PascalCase + full audit	.ForAssemblyOf<T>().WithFullAudit()
snake_case + full audit	.ForAssemblyOf<T>().UseSnakeCase().WithFullAudit()
Soft delete only	.ForAssemblyOf<T>().WithSoftDelete()
Audit only	.ForAssemblyOf<T>().WithAuditFields()
Pluralized + full audit	.ForAssemblyOf<T>().UsePluralizedTables().WithFullAudit()
Custom naming	.ForAssemblyOf<T>().UseNamingConvention(new MyConvention())
Override audit column names	.WithAuditFields(cols => { cols.CreatedDate = ...; })
Override soft delete columns	.WithSoftDelete(cols => { cols.IsDeleted = ...; })
Lazy loading validation	.WithFullAudit().WithLazyLoadingValidation()
Specific types only	EntityConventionBuilder.ForTypes(typeof(A), typeof(B))
Specific assembly	EntityConventionBuilder.ForAssembly(assembly)